



Docker Deployment Guide

This guide explains how to deploy the Vlagalabs-site application using Docker and Docker Compose.



Prerequisites

- Docker Engine 20.10+ ([Install Docker](https://docs.docker.com/get-docker/) (https://docs.docker.com/get-docker/))
- Docker Compose v2.0+ ([Install Docker Compose](https://docs.docker.com/compose/install/) (https://docs.docker.com/compose/install/))
- At least 2GB of available RAM
- At least 5GB of available disk space



Quick Start

1. Clone the Repository

```
git clone https://github.com/atomotavio/Vlagalabs-site.git
cd Vlagalabs-site
```

2. Configure Environment Variables

Copy the example environment file and customize it:

```
cp .env.example .env
```

Edit `.env` and set your values:

```
# REQUIRED: Change this to a secure secret key
PAYLOAD_SECRET=your-super-secret-key-minimum-32-characters

# Database credentials (optional, uses defaults if not set)
POSTGRES_USER=vlagalabs
POSTGRES_PASSWORD=your-secure-password
POSTGRES_DB=vlagalabs

# Application URL
NEXT_PUBLIC_SERVER_URL=http://localhost:3000
```

3. Build and Start Services

```
docker-compose up -d
```

This command will:

- ☒ Build the Next.js application Docker image
- ☒ Pull the PostgreSQL 16 image
- ☒ Create and start both containers
- ☒ Run database migrations automatically
- ☒ Set up volumes for data persistence

4. Access the Application

- **Frontend:** `http://localhost:3000`
- **Payload CMS Admin:** `http://localhost:3000/admin`

On first run, you'll need to create an admin user through the Payload CMS interface.

Docker Architecture

Services

1. app - Next.js Application

- **Base Image:** `node:20-alpine`
- **Exposed Port:** 3000
- **Features:**
 - Multi-stage build for optimization
 - Automatic Prisma and Payload migrations on startup
 - Media uploads persisted in Docker volume
 - Health check dependency on PostgreSQL

2. postgres - PostgreSQL Database

- **Image:** `postgres:16-alpine`
- **Exposed Port:** 5432
- **Features:**
 - Data persisted in Docker volume
 - Health checks for reliable startup
 - Configurable credentials via environment variables

Volumes

- `postgres_data` : Persists PostgreSQL database files
- `media_uploads` : Persists uploaded media files (images, documents)

Docker Commands

View Running Containers

```
docker-compose ps
```

View Logs

```
# All services
docker-compose logs -f

# Specific service
docker-compose logs -f app
docker-compose logs -f postgres
```

Stop Services

```
docker-compose down
```

Stop and Remove Volumes (⚠ Deletes all data)

```
docker-compose down -v
```

Rebuild Application

```
# Rebuild and restart
docker-compose up -d --build

# Force rebuild without cache
docker-compose build --no-cache
docker-compose up -d
```

Execute Commands Inside Container

```
# Access app container shell
docker-compose exec app sh

# Run Prisma migrations manually
docker-compose exec app npx prisma migrate deploy

# Generate Payload types
docker-compose exec app npm run generate:types

# Access PostgreSQL CLI
docker-compose exec postgres psql -U vlagalabs -d vlagalabs
```

Configuration

Environment Variables

Variable	Description	Default	Required
POSTGRES_USER	PostgreSQL user-name	vlagalabs	No
POSTGRES_PASSWORD	PostgreSQL password	vlagalabs_password	Yes*
POSTGRES_DB	PostgreSQL database name	vlagalabs	No
DATABASE_URL	Full database connection string	Auto-generated	No
PAYLOAD_SECRET	Secret key for Payload CMS	your-super-secret-key...	Yes
NEXT_PUBLIC_SERVER_URL	Public URL of the application	http://localhost:3000	Yes

*Should be changed in production

Port Configuration

To change the exposed ports, edit `docker-compose.yml` :

```
services:
  app:
    ports:
      - "8080:3000" # Maps host port 8080 to container port 3000
```

Database Connection from Host

To connect to PostgreSQL from your host machine:

```
psql -h localhost -p 5432 -U vlagalabs -d vlagalabs
```

Or use a GUI tool like pgAdmin, DBeaver, or TablePlus with:

- Host: localhost
- Port: 5432
- User: vlagalabs (or your custom value)
- Password: (from .env)
- Database: vlagalabs (or your custom value)

Production Deployment

Security Considerations

1. Change Default Credentials

env

```
POSTGRES_PASSWORD=use-a-very-strong-random-password
```

```
PAYLOAD_SECRET=use-a-minimum-32-character-random-string
```

2. Use Environment-Specific URLs

env

```
NEXT_PUBLIC_SERVER_URL=https://yourdomain.com
```

3. Don't Expose PostgreSQL Port

Remove or comment out the ports section in docker-compose.yml:

yaml

```
postgres:
```

```
# ports:
```

```
#   - "5432:5432" # Remove this in production
```

4. Set up SSL/TLS

Use a reverse proxy (Nginx, Caddy, Traefik) with SSL certificates.

Using Docker with Reverse Proxy

Example Nginx configuration:

```
server {
    listen 80;
    server_name yourdomain.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

Backup and Restore

Backup Database

```
# Create backup
docker-compose exec postgres pg_dump -U vlagalabs vlagalabs > backup.sql

# Or with Docker volume
docker run --rm -v vlagalabs-site_postgres_data:/data -v $(pwd):/backup alpine \
    tar czf /backup/postgres-backup.tar.gz -C /data .
```

Restore Database

```
# Restore from SQL dump
docker-compose exec -T postgres psql -U vlagalabs vlagalabs < backup.sql

# Or restore volume
docker run --rm -v vlagalabs-site_postgres_data:/data -v $(pwd):/backup alpine \
  tar xzf /backup/postgres-backup.tar.gz -C /data
```

Backup Media Files

```
# Create media backup
docker run --rm -v vlagalabs-site_media_uploads:/data -v $(pwd):/backup alpine \
  tar czf /backup/media-backup.tar.gz -C /data .

# Restore media backup
docker run --rm -v vlagalabs-site_media_uploads:/data -v $(pwd):/backup alpine \
  tar xzf /backup/media-backup.tar.gz -C /data
```



Troubleshooting

Container Fails to Start

Check logs:

```
docker-compose logs app
docker-compose logs postgres
```

Common issues:

- Database not ready: Wait a few seconds and check `docker-compose logs postgres`
- Port already in use: Change port mapping in `docker-compose.yml`
- Insufficient memory: Increase Docker memory limit

Database Connection Errors

Symptoms:

```
Error: P1001: Can't reach database server
```

Solutions:

1. Verify PostgreSQL is running: `docker-compose ps`
2. Check database logs: `docker-compose logs postgres`
3. Verify DATABASE_URL is correct in app container:


```
bash
docker-compose exec app printenv DATABASE_URL
```

Build Failures

Clean build:

```
docker-compose down
docker system prune -a
docker-compose build --no-cache
docker-compose up -d
```

Migration Errors

Reset migrations:

```
# ⚠ This will delete all data
docker-compose down -v
docker-compose up -d
```

Run migrations manually:

```
docker-compose exec app npx prisma migrate deploy
docker-compose exec app npm run payload migrate
```

Permission Issues

Fix media folder permissions:

```
docker-compose exec app chown -R nextjs:nodejs /app/media
```



Monitoring

Resource Usage

```
# View resource usage
docker stats

# View specific container
docker stats vlagalabs-app
```

Health Checks

```
# Check PostgreSQL health
docker-compose exec postgres pg_isready -U vlagalabs

# Check app health (if endpoint exists)
curl http://localhost:3000/api/health
```

Updates and Maintenance

Update Application

```
# Pull latest code
git pull origin main

# Rebuild and restart
docker-compose up -d --build
```

Update Docker Images

```
# Pull latest base images
docker-compose pull

# Rebuild with new base images
docker-compose up -d --build
```

Additional Resources

- [Docker Documentation](https://docs.docker.com/) (https://docs.docker.com/)
- [Docker Compose Documentation](https://docs.docker.com/compose/) (https://docs.docker.com/compose/)
- [Next.js Docker Deployment](https://nextjs.org/docs/deployment#docker-image) (https://nextjs.org/docs/deployment#docker-image)
- [PostgreSQL Docker Hub](https://hub.docker.com/_/postgres) (https://hub.docker.com/_/postgres)
- [Payload CMS Documentation](https://payloadcms.com/docs) (https://payloadcms.com/docs)



Tips

1. **Development Mode:** For development, you may want to mount the source code:

```
```yaml
volumes:
 - ./:/app
 - /app/node_modules
```
```
2. **Environment-specific Configs:** Use `docker-compose.override.yml` for local customizations
3. **Multi-stage Builds:** The Dockerfile uses multi-stage builds to optimize image size
4. **Logging:** Application logs are captured by Docker and can be viewed with `docker-compose logs`

Need help? Open an issue on the GitHub repository or consult the main README.md file.